Module 5 — Building Prediction Endpoints (/predict) for AI Engineers

Learning Objectives

By the end of this module, you will be able to:

- Construct a production-grade **Path Operation** specifically for model inference.
- Design **Request Bodies** using Pydantic to handle complex ML feature sets.
- Implement **Preprocessing** and **Postprocessing** logic within the Path Operation Function.
- Handle **Batch Inference** by accepting and returning lists of data.
- Utilize **Response Models** to filter raw model outputs into clean, standardized JSON.

The Heart of AI Services: The `/predict` Endpoint

In AI Engineering, the **Prediction Endpoint** is the specific URL (path) that allows external systems to "talk" to your model. While a notebook allows you to run `model.predict(X)`, a production endpoint allows a mobile app or another server to send data to your model and receive a result in real-time.

The Inference Data Flow

Unlike simple web pages, AI endpoints follow a strict numerical pipeline:

1. **Client**: Sends a **POST** request containing raw features in JSON format.
2. **API Schema**: FastAPI validates the types (e.g., ensuring an "age" feature is an `int`).
3. **Preprocessing**: The function converts JSON strings or lists into the format your model expects (e.g., a NumPy array or Torch tensor).
4. **Model Inference**: The loaded model processes the data.
5. **Postprocessing**: The raw output (like a probability of `0.98`) is converted into a human-readable label (like `"Fraud"`).
6. **Output**: A **Response Model** ensures the client only sees the final prediction and confidence score, hiding internal model messiness.

Core Concepts for Inference

1. Why use POST for Inference?

While **GET** is used for simple health checks, **POST** is the industry standard for inference.

- **Why this exists**: Model inputs (feature vectors, images, or long text) are often too large to fit in a URL. POST allows you to send a **Request Body**.
- **Where it is used**: In every real-time prediction service, such as **Uber's** Ludwig server or **Microsoft's** ML services.
- **What breaks**: If you use GET, your system will crash if the feature vector is too long, and sensitive data (like medical records) will be visible in server logs.

2. Preprocessing: The Training-Production Bridge

Your model was trained on specific data shapes. If you trained on normalized values (0 to 1), your API must normalize incoming data before the model sees it.

- **Why this exists**: To ensure "Data Consistency." A model trained on centimeters will give wrong predictions if the API accepts inches.
- **Where it is used**: Converting a user's categorical input (e.g., `"High Risk"`) into a numerical flag (e.g., `1`).
- **What breaks**: The model produces "garbage" results or crashes due to **Shape Mismatches**.

3. Handling Single vs. Batch Predictions

- **Single Prediction**: One input, one output. Good for mobile apps.
- **Batch Prediction**: A **List** of inputs processed in one go.
- **Why this exists**: Batching is computationally more efficient for deep learning models running on GPUs.
- **Where it is used**: Processing multiple transactions at once for a **Fraud Detection API**.

Code Examples

Example 1: Numeric ML Model Prediction (Regression)

This example simulates a **Price Prediction Service**.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# 1. Request Schema (Features)
class HouseFeatures(BaseModel):
    sq_ft: int
    location_score: float

# 2. Response Schema (Cleaned Output)
class PriceResponse(BaseModel):
    estimated_price: float
```

```
@app.post("/predict", response_model=PriceResponse) # (A)
def predict_price(request: HouseFeatures): # (B)
    # (C) Preprocessing: Simulation of a math operation
    # Real AI logic: raw_val = model.predict([[request.sq_ft,
request.location_score]])
    raw_val = request.sq_ft * request.location_score * 100

    # (D) Postprocessing: Formatting the output
    return {"estimated_price": raw_val}
```

Line-by-Line Explanation:

- **(A)** **`@app.post(...)`**: The **Path Operation Decorator**. It defines the endpoint `/predict` and uses `response_model` to format the output.
- **(B)** **`request: HouseFeatures`**: FastAPI automatically parses the incoming JSON and validates it against our `BaseModel`.
- **(C) Preprocessing**: We access the features directly using dot notation (`request.sq_ft`). In a real system, you would pass these into your model here.
- **(D) Return Statement**: We return a dictionary. Because of the `response_model`, FastAPI ensures this matches the `PriceResponse` exactly.

Example 2: Batch Inference Endpoint

Used for processing multiple items at once to save time.

```
from typing import List # (A)
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Transaction(BaseModel):
    amount: float
    is_international: bool

@app.post("/predict-batch")
def predict_batch(items: List[Transaction]): # (B)
    results = []
    for item in items: # (C)
        # Simulation: Logic for fraud detection
        is_fraud = item.amount > 10000 and item.is_international
        results.append({"fraud_risk": is_fraud})
    return results # (D)
```

Line-by-Line Explanation:

- **(A)** **`from typing import List`**: Essential for defining collections of data.
- **(B)** **`items: List[Transaction]`**: Tells FastAPI to expect a **JSON Array** of objects.
- **(C)** **`for item in items`**: Standard Python loop to process each record in the batch.
- **(D)** **`return results`**: Returns a list of predictions back to the client.

Scenario Discussions

Scenario A: Fraud Detection API (Classification)

In a **Classification** scenario, the model usually returns probabilities.

- **The Problem**: A raw model might return `[0.02, 0.98]`. A user doesn't know what that means.
- **The AI Engineer's Task**: Postprocess that array into `{"label": "Fraud", "confidence": 0.98}` using a **Response Model**.

Scenario B: Price Prediction (Regression)

In **Regression**, the output is a continuous number.

- **The Task**: Ensure the output is rounded and properly typed as a `float` so the consuming system (like a dashboard) doesn't break.

Common Mistakes AI Engineers Make

- **Forgetting Response Models**: Returning raw `NumPy` arrays or `Tensors`. These objects are **not JSON serializable** and will cause an "Internal Server Error".
- **Preprocessing Mismatch**: Applying a scaler during training but forgetting to apply the same scaler in the FastAPI function. This leads to wildly incorrect predictions.
- **Using GET for Data**: Passing features in query parameters. This makes it impossible to send high-dimensional data (like a 512-value embedding).
- **Ignoring Validation Errors**: If a user sends a string for a numeric feature, FastAPI handles it with a 422 error, but AI engineers often try to manually re-validate this inside the function, adding unnecessary code.

Assignment: The "Model Identity" Predictor

1. Create a Pydantic model `ModelInput` with three fields: `feature_a` (float), `feature_b` (float), and `category` (string).
2. Create a Response Model `ModelOutput` that returns a `prediction_score` (float) and a `status` (string).
3. Implement a `/predict` POST endpoint.
4. If `category` is `"test"`, the `prediction_score` should be `feature_a + feature_b`.
5. If `category` is anything else, return a score of `0.0`.
6. Use the `/docs` UI to verify it works with different inputs.

Mini Project: Production Inference API

Goal: Build an "Image Metadata Classifier" (without actual image files).

- Create a schema that accepts image metadata: `width`, `height`, `channels`, and `file_format`.
- Implement a `/predict` endpoint that:
  - **Preprocesses**: Checks if `width * height` exceeds a "limit."
  - **Inference**: If the "limit" is exceeded, it classifies the image as `"High Resolution"`. Otherwise, `"Standard Resolution"`.
  - **Postprocessing**: Returns the classification label and the calculated `total_pixels`.
- Ensure all outputs are filtered through a `Response Model`.

Readiness Checklist

- [ ] Can I explain why **POST** is used for sending feature vectors?
- [ ] Do I know how to use **List[Schema]** to handle batches?
- [ ] Do I understand the role of **Preprocessing** in maintaining model accuracy?
- [ ] Can I use the **Swagger UI (/docs)** to test my `/predict` endpoint?
- [ ] Am I able to use **Response Models** to clean up my model's output?